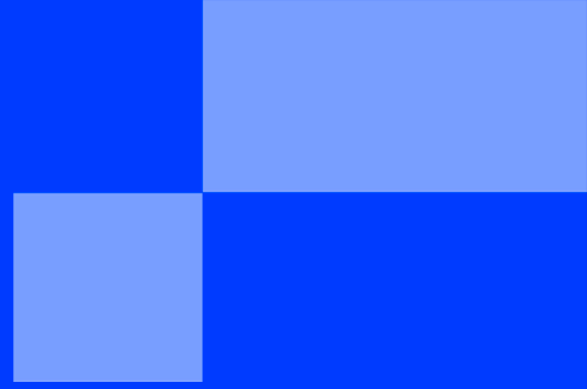




SUMMER SCHOOL

< PART 2 - INTRODUCTION TO REACT />

SUMMER SCHOOL

< WEB DEVELOPMENT: THE LEGO BLOCKS />

"Any fool can write code that a computer can understand. Good programmers write code that humans can understand."

— **Martin Fowler**

The Case

Your static HTML prototype was a massive success. The Editor-in-Chief loves the clean layout and the three-column article grid. However, the Chief Technology Officer (CTO) has raised a red flag.

Currently, if the journalists want to publish 50 articles, developers have to copy and paste the `<article>` block 50 times in the `index.html` file. If the design team decides to change the "Read the full story" button color, developers have to manually change it 50 times. This is an unmaintainable nightmare.

Your mission today is to modernize the newspaper's architecture. You will rebuild yesterday's exact layout, but this time using **React**. You will break the user interface down into reusable Lego blocks called "Components".

Theory: The React Paradigm

React is a JavaScript library built by Facebook. It changes how we think about building websites. Instead of writing one giant HTML file, we write small, independent pieces of UI and assemble them.

1. Components

A Component is simply a JavaScript function that returns a piece of user interface. Always start a component name with a **Capital Letter**.

```
// This is a React Component
function Header() {
  return (
    <header>
      <h1>Welcome to the Laboratory</h1>
    </header>
  );
}
```

2. JSX: HTML inside JavaScript

Look at the code above. It looks like HTML, but it is inside a JavaScript file! This is called **JSX**. It has strict rules:

- **Rule 1:** You can only return **one** single parent element. If you have siblings, wrap them in a generic `<div>` or an empty fragment `<> </>`.
- **Rule 2:** Because `class` is a reserved keyword in JavaScript, you **MUST** use `className` to apply your CSS classes.

```
// Incorrect HTML syntax in React: <div class="menu">. Correct JSX syntax:
<div className="menu">
```



Tip: Keep your components small and focused. A component should do one thing, and do it well.

```
function Footer() {
  return (
    <footer>
      <p>© The Laboratory</p>
    </footer>
  );
}
```

3. Props (Properties)

The real power of React is reusability. Instead of hardcoding the text inside your article card, you can pass variables to your component. These are called `props`.

```
// The component expects a "title" and an "excerpt"
function ArticleCard(props) {
  return (
    <article className="card">
      <h2>{props.title}</h2>
      <p>{props.excerpt}</p>
      <a href="#">Read the full story</a>
    </article>
  );
}
```

When you use the component, you pass the data just like HTML attributes:

```
<ArticleCard title="Campus Life Returns" excerpt="Students are resting..." />
<ArticleCard title="Tech Advances" excerpt="New technologies emerge..." />
```



Tip: You can also use **destructuring** to make your code cleaner:

```
function ArticleCard({ title, excerpt }) {
  return (
    <article className="card">
      <h2>{title}</h2>
      <p>{excerpt}</p>
      <a href="#">Read the full story</a>
    </article>
  );
}
```

Project Requirements

You will recreate your layout using React components. We want you to focus purely on the React logic, so we are providing you with a complete stylesheet.

Step 1: Initialize the React Application

You cannot just open an `index.html` file anymore. React requires a build environment. Open your terminal in your project folder and run the modern build tool, Vite:

```
npm create vite@latest newspaper-react -- --template react #Initialize a new React project
cd newspaper-react #Move into the project folder
npm install #Install dependencies
npm run dev #Start the project server
```

Open the URL provided in your terminal (usually `http://localhost:5173`). You are now looking at a live React environment!

Step 2: Clean the Boilerplate & Add CSS

Vite generates a lot of default code. Let's start fresh and make it look like a real newspaper immediately.

1. Open the `src/` folder.
2. Delete the `App.css` file completely.
3. Open `index.css`, delete everything inside, and paste this ready-to-use CSS:

```
body { font-family: 'Arial', sans-serif; background-color: #f8fafc; color: #0f172a; margin: 0; }
.app-container { max-width: 1000px; margin: 0 auto; padding: 20px; }
header { display: flex; justify-content: space-between; align-items: center; padding: 20px 0;
border-bottom: 2px solid #e2e8f0; margin-bottom: 40px; }
header h1 { color: #1e293b; margin: 0; }
nav { display: flex; gap: 20px; }
nav a { text-decoration: none; color: #3b82f6; font-weight: bold; }
.hero { background-color: #1e293b; color: white; padding: 50px 30px; border-radius: 12px;
margin-bottom: 40px; text-align: center; }
.article-grid { display: flex; gap: 20px; margin-bottom: 40px; }
.card { background-color: white; padding: 25px; border-radius: 8px; box-shadow: 0 4px 6px rgba(0,
0, 0, 0.05); flex: 1; }
.card h2 { margin-top: 0; color: #0f172a; font-size: 1.2rem; }
.card a { display: inline-block; margin-top: 15px; color: #3b82f6; text-decoration: none;
font-weight: bold; }
footer { text-align: center; padding: 40px 0; border-top: 2px solid #e2e8f0; color: #64748b; }
```

4. Open `App.jsx` and delete everything inside the `return ()` statement.

Step 3: Componentizing the Page

Inside your `src/` folder, create a new folder called `components/`. You must build the pieces of your newspaper. Create the following files:

1. **Header.jsx**: Must return a `<header>` tag containing your `<h1>` and the `<nav>` menu.
2. **Hero.jsx**: Must return a `<section className="hero">` containing your main breaking news article (an `<h2>` and a `<p>`).
3. **ArticleCard.jsx**: Must return a single `<article className="card">` block. It must accept `title` and `excerpt` as props so the content is dynamic.
4. **ArticleGrid.jsx**: Must act as a `<section className="article-grid">` container. Inside it, you will call your `<ArticleCard />` component at least three times, passing different text via props for each card.
5. **Footer.jsx**: Must return a `<footer>` containing the copyright information.

Step 4: The Final Assembly

Go back to your `App.jsx` file. This is the root of your application. Import all your newly created components at the top, and assemble them in the correct logical order inside the main return:

```
import Header from './components/Header';
import Hero from './components/Hero';
import ArticleGrid from './components/ArticleGrid';
import Footer from './components/Footer';

function App() {
  return (
    <div className="app-container">
      <Header />
      <main>
        <Hero />
        <ArticleGrid />
      </main>
      <Footer />
    </div>
  );
}

export default App;
```

Your project should look better than ever! Save your files and check your browser. You should see a clean, modern newspaper layout, fully built with React components.

v1

{EPITECH}